

Faculty of Engineering - Alexandria University

Computer and Communications Engineering

Lab 4 - Weather Stations Monitoring

Technical Report

Distributed IoT Weather Monitoring

10 Weather Stations → Kafka + ZooKeeper

Rain Processor + Central Station → Persistent PostgreSQL

Course: Net Centric Computing
Academic Year: Fall 2025–2026
Date: August 24, 2026

Team Members

Name	ID
Ahmed Walaaeldin	8625
Mazen Adel	8634
Marwan Ahmed	8824
Ahmed Ibrahim	8672

Contents

1	Introduction	3
2	System Architecture	3
2.1	High-Level Design	3
2.2	Repository Organization	4
3	Weather Station Mock - Part A	6
3.1	Reading Generation	6
3.2	Battery Status Distribution	6
3.3	Simulated Weather and Message Drops	7
4	Kafka Integration - Part B	7
4.1	Producer Configuration	7
4.2	Partitioning and Ordering	8
5	Rain Detection - Part C	9
5.1	Kafka Streams Processor API	9
5.2	Processor API Versus DSL	9
6	Central Station - Part D	10
6.1	Concurrent Consumers	10
6.2	Batch Persistence	10
6.3	Delivery Semantics and Idempotency	11
6.4	Database Schema	11
7	Historical Weather Analysis - Part E	12
7.1	Battery Status Distribution Per Station	12
7.2	Dropped Messages Per Station	13
7.3	Latest Weather Status Per Station	14
7.4	Rain Alert Cross-Check	15
8	Kubernetes Deployment - Part F	16
8.1	Deployment Environment and Containerization	16
8.2	Kubernetes Resources	16
8.3	Kafka and ZooKeeper Coordination	18
8.4	Weather Station StatefulSet	19
8.5	PostgreSQL Persistent Storage	19
8.6	Database Persistence Verification	20
8.7	Configuration and Secrets	21
8.8	Runtime Evidence	21
9	Challenges and Lessons Learned	23
10	Conclusion	23

List of Figures

1	Implemented Kubernetes architecture with ZooKeeper-backed Kafka and persistent PostgreSQL storage.	3
2	Project repository structure showing application modules, Dockerfiles, SQL scripts, and Kubernetes manifests.	5
3	Weather station logs showing generated readings and intentionally dropped messages.	7
4	Weather message consumed directly from the Kafka weather-readings topic.	8
5	Kafka topics for weather readings and rain alerts, each configured with ten partitions.	9
6	Rain processor detecting humidity values greater than 70%.	10
7	Rain-alert message consumed from the Kafka rain-alerts topic.	10
8	Central station consuming Kafka messages and persisting weather readings and rain alerts.	12
9	Measured battery-status distribution for all ten weather stations.	13
10	Estimated dropped-message count and drop percentage for each weather station.	14
11	Latest weather reading retrieved directly from PostgreSQL for each station.	14
12	Per-station cross-check between high-humidity readings and their corresponding rain alerts.	16
13	Kubernetes pods for ZooKeeper, Kafka, PostgreSQL, ten weather stations, the rain processor, and the central station.	17
14	Kubernetes Services, application Deployments, topic-initialization Job, and the ten-replica Weather Station StatefulSet.	18
15	Kafka ZooKeeper connection and successful registration of broker 1.	19
16	Bound PostgreSQL PersistentVolume and PersistentVolumeClaim mounted by the PostgreSQL Deployment.	20
17	PostgreSQL pod-replacement test confirming that weather readings and rain alerts survived the restart.	20
18	Kubernetes ConfigMap values and database Secret metadata without exposing credential values.	21
19	Weather station pod logs showing generated readings and simulated message drops.	22
20	Rain Processor running in Kubernetes and detecting humidity values greater than 70%.	22
21	Central Station logs showing JDBC batch persistence inside Kubernetes.	22
22	PostgreSQL query results obtained from the Kubernetes deployment.	23

1 Introduction

The objective of this lab is to implement a distributed weather monitoring system that models an Internet of Things (IoT) stream-processing pipeline. Multiple weather stations generate sensor readings, Apache Kafka provides asynchronous transport, a Kafka Streams processor detects rain conditions, and a central station stores historical data in a relational database.

Ten independent weather stations generate one sample per second. Each sample contains a station identifier, a per-station sequence number, battery status, Unix timestamp, humidity, temperature, and wind speed. Approximately 10% of generated samples are intentionally dropped. Battery status is sampled using probabilities of 30% low, 40% medium, and 30% high.

Weather readings are published to the `weather-readings` Kafka topic. A Kafka Streams processor identifies humidity values greater than 70% and publishes corresponding messages to `rain-alerts`. The central station consumes both topics and persists the resulting records in PostgreSQL using batched database writes.

The complete system runs on Docker Desktop Kubernetes for macOS. The deployment includes a ZooKeeper-backed Kafka broker, ten weather station pods, a rain processor, a central station, and a PostgreSQL database backed by a 5 GiB PersistentVolume and PersistentVolumeClaim. Database persistence was verified by replacing the PostgreSQL pod and confirming that all previously stored readings and rain alerts remained available.

2 System Architecture

2.1 High-Level Design

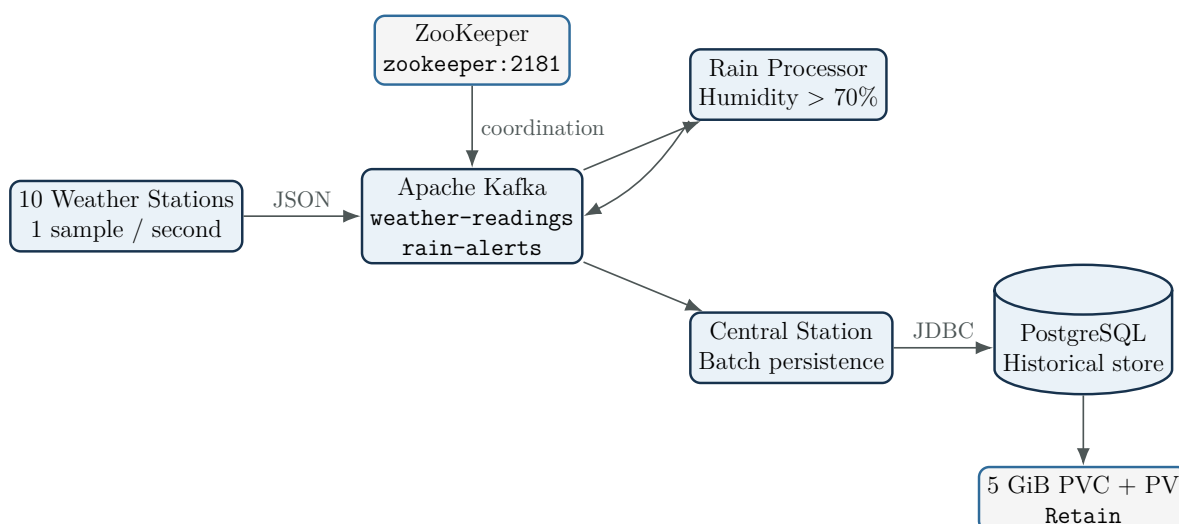


Figure 1: Implemented Kubernetes architecture with ZooKeeper-backed Kafka and persistent PostgreSQL storage.

The architecture has three main stages:

1. **Data acquisition:** ten weather station services generate simulated sensor readings and publish them to Kafka.
2. **Data processing:** Kafka buffers the readings, while the rain processor produces alerts for humidity values above 70%.
3. **Data persistence and analysis:** the central station stores weather readings and rain alerts in PostgreSQL for historical analysis.

Kafka separates producers from consumers and provides buffering, replay capability, per-key ordering, and independent processing of the two message streams. ZooKeeper coordinates the Kafka broker, while the PostgreSQL PersistentVolume separates database storage from the database pod lifecycle.

2.2 Repository Organization

The implementation is organized as a Maven multi-module project. The `common` module contains shared message classes and configuration helpers. Separate application modules implement the weather stations, rain processor, and central station. Kubernetes manifests, Dockerfiles, and SQL scripts are stored alongside the application source code.

```
(base) ahmedwalaa@192 distributed-weather-monitoring-system % tree -L 2 -I '.git/target/images'
.
├── analysis_queries.sql
├── central-station
│   ├── Dockerfile
│   ├── pom.xml
│   └── src
├── cloud
│   ├── central.env.example
│   ├── CLOUD-DEPLOYMENT.md
│   ├── docker-compose.central.yml
│   └── stations-compose.sh
├── common
│   ├── pom.xml
│   └── src
├── context.md
├── docker-compose.yml
├── docs
│   ├── CLOUD-DEPLOYMENT.md
│   ├── Lab4-NetCentric.pdf
│   └── REPORT.md
├── k8s
│   ├── 00-namespace-config.yaml
│   ├── 01-zookeeper.yaml
│   ├── 02-kafka.yaml
│   ├── 02-postgres-storage.yaml
│   ├── 02-postgres.yaml
│   ├── 03-weather-stations.yaml
│   ├── 04-rain-processor.yaml
│   ├── 05-central-station.yaml
│   └── deploy.sh
├── pom.xml
├── rain-processor
│   ├── Dockerfile
│   ├── pom.xml
│   └── src
├── README.md
├── report
│   ├── main.pdf
│   └── main.tex
├── sql
│   ├── analysis_queries.sql
│   └── schema.sql
└── weather-station
    ├── Dockerfile
    ├── pom.xml
    └── src

14 directories, 32 files
```

Figure 2: Project repository structure showing application modules, Dockerfiles, SQL scripts, and Kubernetes manifests.

Table 1: Main repository components.

Path	Purpose
<code>common/</code>	Shared JSON models, Kafka topic names, and environment-based configuration helpers.
<code>weather-station/</code>	Parts A and B: reading generation and Kafka production.
<code>rain-processor/</code>	Part C: stateful Kafka Streams rain detection.
<code>central-station/</code>	Part D: Kafka consumers, JDBC batching, and database initialization.
<code>sql/</code>	Part E: relational schema and historical analysis queries.
<code>k8s/</code>	Part F: ZooKeeper, Kafka, PostgreSQL, persistent storage, application manifests, and deployment script.
<code>report/</code>	LaTeX report source, generated PDF, and supporting screenshots.

3 Weather Station Mock - Part A

3.1 Reading Generation

Each weather station generates one `WeatherMessage` every second. Its identity is supplied through the `STATION_ID` environment variable or derived from its Kubernetes StatefulSet pod ordinal. Pod names `weather-station-0` through `weather-station-9` therefore correspond to logical station IDs 1 through 10.

The generated weather message has the following structure:

Listing 1: Weather status JSON schema.

```

1 {
2   "station_id": 1,
3   "s_no": 1,
4   "battery_status": "low",
5   "status_timestamp": 1681521224,
6   "weather": {
7     "humidity": 35,
8     "temperature": 100,
9     "wind_speed": 13
10  }
11 }
```

The sequence number increases for every generated sample before the message-drop decision. Consequently, intentionally dropped samples leave observable gaps in the stored sequence numbers.

3.2 Battery Status Distribution

A uniformly distributed random value $p \in [0, 1)$ determines the battery state:

- `low` when $p < 0.30$;

- medium when $0.30 \leq p < 0.70$;
- high when $p \geq 0.70$.

These thresholds produce the required theoretical distribution of 30% low, 40% medium, and 30% high.

3.3 Simulated Weather and Message Drops

Humidity is generated between 0 and 100%, temperature between 20 and 120 degrees Fahrenheit, and wind speed between 0 and 50 km/h. A second independent random draw drops approximately 10% of generated samples. Dropped sequence numbers are not reused.

```
(base) ahmedwalee@ahmeds-air distributed-weather-monitoring-system % docker compose logs --tail=30 weather-station
weather-station-1 | [station-1] sent=1828 dropped=139 last=(station_id:1,"s_no":1159,"battery_status":"medium","status_timestamp":1787493422,"weather":{"humidity":24,"temperature":45,"wind_speed":12})
weather-station-1 | [station-1] dropped s_no=1171 (total dropped=140)
weather-station-1 | [station-1] sent=1958 dropped=141 last=(station_id:1,"s_no":1191,"battery_status":"medium","status_timestamp":1787493454,"weather":{"humidity":61,"temperature":119,"wind_speed":39})
weather-station-1 | [station-1] sent=1888 dropped=145 last=(station_id:1,"s_no":1225,"battery_status":"high","status_timestamp":1787493488,"weather":{"humidity":59,"temperature":58,"wind_speed":15})
weather-station-1 | [station-1] sent=1116 dropped=147 last=(station_id:1,"s_no":1257,"battery_status":"low","status_timestamp":1787493520,"weather":{"humidity":89,"temperature":61,"wind_speed":28})
weather-station-1 | [station-1] dropped s_no=1171 (total dropped=148)
weather-station-1 | [station-1] sent=1148 dropped=153 last=(station_id:1,"s_no":1293,"battery_status":"medium","status_timestamp":1787493556,"weather":{"humidity":61,"temperature":88,"wind_speed":24})
weather-station-1 | [station-1] sent=1178 dropped=157 last=(station_id:1,"s_no":1327,"battery_status":"medium","status_timestamp":1787493590,"weather":{"humidity":26,"temperature":87,"wind_speed":12})
weather-station-1 | [station-1] sent=1298 dropped=158 last=(station_id:1,"s_no":1358,"battery_status":"medium","status_timestamp":1787493621,"weather":{"humidity":48,"temperature":65,"wind_speed":44})
weather-station-1 | [station-1] dropped s_no=1172 (total dropped=148)
weather-station-1 | [station-1] sent=1238 dropped=160 last=(station_id:1,"s_no":1390,"battery_status":"high","status_timestamp":1787493653,"weather":{"humidity":60,"temperature":31,"wind_speed":29})
weather-station-1 | [station-1] sent=1264 dropped=161 last=(station_id:1,"s_no":1421,"battery_status":"high","status_timestamp":1787493684,"weather":{"humidity":20,"temperature":48,"wind_speed":8})
weather-station-1 | [station-1] sent=1298 dropped=162 last=(station_id:1,"s_no":1452,"battery_status":"low","status_timestamp":1787493715,"weather":{"humidity":90,"temperature":97,"wind_speed":41})
weather-station-1 | [station-1] sent=1328 dropped=166 last=(station_id:1,"s_no":1486,"battery_status":"low","status_timestamp":1787493749,"weather":{"humidity":29,"temperature":35,"wind_speed":8})
weather-station-1 | [station-1] dropped s_no=1151 (total dropped=170)
weather-station-1 | [station-1] sent=1358 dropped=170 last=(station_id:1,"s_no":1520,"battery_status":"high","status_timestamp":1787493783,"weather":{"humidity":73,"temperature":54,"wind_speed":13})
weather-station-1 | [station-1] sent=1388 dropped=177 last=(station_id:1,"s_no":1557,"battery_status":"low","status_timestamp":1787493820,"weather":{"humidity":32,"temperature":91,"wind_speed":8})
weather-station-1 | [station-1] sent=1416 dropped=178 last=(station_id:1,"s_no":1588,"battery_status":"low","status_timestamp":1787493851,"weather":{"humidity":29,"temperature":37,"wind_speed":13})
weather-station-1 | [station-1] dropped s_no=1619 (total dropped=180)
weather-station-1 | [station-1] sent=1448 dropped=180 last=(station_id:1,"s_no":1620,"battery_status":"medium","status_timestamp":1787493883,"weather":{"humidity":17,"temperature":28,"wind_speed":42})
weather-station-1 | [station-1] sent=1478 dropped=182 last=(station_id:1,"s_no":1652,"battery_status":"high","status_timestamp":1787493915,"weather":{"humidity":60,"temperature":181,"wind_speed":3})
weather-station-1 | [station-1] sent=1508 dropped=187 last=(station_id:1,"s_no":1687,"battery_status":"medium","status_timestamp":1787493950,"weather":{"humidity":48,"temperature":195,"wind_speed":21})
weather-station-1 | [station-1] sent=1538 dropped=189 last=(station_id:1,"s_no":1719,"battery_status":"medium","status_timestamp":1787493983,"weather":{"humidity":85,"temperature":20,"wind_speed":16})
weather-station-1 | [station-1] dropped s_no=1731 (total dropped=190)
weather-station-1 | [station-1] sent=1568 dropped=190 last=(station_id:1,"s_no":1753,"battery_status":"medium","status_timestamp":1787494017,"weather":{"humidity":15,"temperature":40,"wind_speed":37})
weather-station-1 | [station-1] dropped s_no=1789 (total dropped=200)
weather-station-1 | [station-1] sent=1598 dropped=200 last=(station_id:1,"s_no":1790,"battery_status":"high","status_timestamp":1787494054,"weather":{"humidity":4,"temperature":91,"wind_speed":10})
weather-station-1 | [station-1] sent=1628 dropped=201 last=(station_id:1,"s_no":1823,"battery_status":"high","status_timestamp":1787494087,"weather":{"humidity":33,"temperature":162,"wind_speed":23})
weather-station-1 | [station-1] sent=1658 dropped=205 last=(station_id:1,"s_no":1855,"battery_status":"high","status_timestamp":1787494119,"weather":{"humidity":89,"temperature":64,"wind_speed":3})
weather-station-1 | [station-1] sent=1688 dropped=208 last=(station_id:1,"s_no":1888,"battery_status":"medium","status_timestamp":1787494152,"weather":{"humidity":3,"temperature":44,"wind_speed":18})
(base) ahmedwalee@ahmeds-air distributed-weather-monitoring-system %
```

Figure 3: Weather station logs showing generated readings and intentionally dropped messages.

4 Kafka Integration - Part B

4.1 Producer Configuration

The weather station uses the Java Kafka Producer API to publish serialized JSON messages to the `weather-readings` topic. The principal producer settings are summarized below.

Table 2: Weather station Kafka producer configuration.

Property	Value	Reason
Bootstrap servers	Environment configured	Supports the deployed Kubernetes Kafka endpoint.
Key serializer	StringSerializer	Uses the station ID as the message key.
Value serializer	StringSerializer	Serializes weather messages as JSON strings.
acks	all	Requires acknowledgement from the available broker replicas.
Idempotence	enabled	Prevents duplicate producer writes caused by retries.
Retries	3	Handles transient Kafka communication failures.
linger.ms	20 ms	Allows short producer-side batching.
Compression	gzip	Reduces the message payload transmitted over the network.

4.2 Partitioning and Ordering

The Kafka message key is the station ID. Readings from an individual station are therefore routed consistently to the same partition, preserving their relative order. The deployment creates ten partitions for both `weather-readings` and `rain-alerts`. Distinct station IDs can still map to the same partition because partition assignment is based on hashing rather than a dedicated partition per station.

```
(base) ahmedwale@bsheds-air distributed-weather-monitoring-system % docker exec -it $(docker ps -qf name=kafka) \
  /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 \
  --topic weather-readings --max-messages 5
The consumer rebalance protocol (KIP-848) is production-ready! Set group.protocol=consumer to try it out. See https://kafka.apache.org/documentation/#consumer_rebalance_protocol
{"station_id":1,"s_no":2078,"battery_status":"medium","status_timestamp":1787494342,"weather":{"humidity":62,"temperature":63,"wind_speed":47}}
{"station_id":9,"s_no":1021,"battery_status":"high","status_timestamp":1787494342,"weather":{"humidity":6,"temperature":87,"wind_speed":34}}
{"station_id":13,"s_no":1027,"battery_status":"medium","status_timestamp":1787494342,"weather":{"humidity":47,"temperature":54,"wind_speed":37}}
{"station_id":16,"s_no":1026,"battery_status":"medium","status_timestamp":1787494343,"weather":{"humidity":1,"temperature":44,"wind_speed":38}}
{"station_id":7,"s_no":1024,"battery_status":"medium","status_timestamp":1787494343,"weather":{"humidity":59,"temperature":76,"wind_speed":11}}
Processed a total of 5 messages
```

Figure 4: Weather message consumed directly from the Kafka weather-readings topic.

```
(base) ahmedwala@192 distributed-weather-monitoring-system % kubectl -n weather exec deploy/kafka -- \
kafka-topics.sh \
--bootstrap-server kafka:9092 \
--describe \
--topic weather-readings
Topic: weather-readings TopicId: P4Zx1oIW6Sr_0zKruBaGg PartitionCount: 10 ReplicationFactor: 1 Configs:
Topic: weather-readings Partition: 0 Leader: 1 Replicas: 1 Isr: 1
Topic: weather-readings Partition: 1 Leader: 1 Replicas: 1 Isr: 1
Topic: weather-readings Partition: 2 Leader: 1 Replicas: 1 Isr: 1
Topic: weather-readings Partition: 3 Leader: 1 Replicas: 1 Isr: 1
Topic: weather-readings Partition: 4 Leader: 1 Replicas: 1 Isr: 1
Topic: weather-readings Partition: 5 Leader: 1 Replicas: 1 Isr: 1
Topic: weather-readings Partition: 6 Leader: 1 Replicas: 1 Isr: 1
Topic: weather-readings Partition: 7 Leader: 1 Replicas: 1 Isr: 1
Topic: weather-readings Partition: 8 Leader: 1 Replicas: 1 Isr: 1
Topic: weather-readings Partition: 9 Leader: 1 Replicas: 1 Isr: 1
(base) ahmedwala@192 distributed-weather-monitoring-system % kubectl -n weather exec deploy/kafka -- \
kafka-topics.sh \
--bootstrap-server kafka:9092 \
--describe \
--topic rain-alerts
Topic: rain-alerts TopicId: v3AUyHBuRCyYZC70S9Gd2g PartitionCount: 10 ReplicationFactor: 1 Configs:
Topic: rain-alerts Partition: 0 Leader: 1 Replicas: 1 Isr: 1
Topic: rain-alerts Partition: 1 Leader: 1 Replicas: 1 Isr: 1
Topic: rain-alerts Partition: 2 Leader: 1 Replicas: 1 Isr: 1
Topic: rain-alerts Partition: 3 Leader: 1 Replicas: 1 Isr: 1
Topic: rain-alerts Partition: 4 Leader: 1 Replicas: 1 Isr: 1
Topic: rain-alerts Partition: 5 Leader: 1 Replicas: 1 Isr: 1
Topic: rain-alerts Partition: 6 Leader: 1 Replicas: 1 Isr: 1
Topic: rain-alerts Partition: 7 Leader: 1 Replicas: 1 Isr: 1
Topic: rain-alerts Partition: 8 Leader: 1 Replicas: 1 Isr: 1
Topic: rain-alerts Partition: 9 Leader: 1 Replicas: 1 Isr: 1
(base) ahmedwala@192 distributed-weather-monitoring-system %
```

Figure 5: Kafka topics for weather readings and rain alerts, each configured with ten partitions.

5 Rain Detection - Part C

5.1 Kafka Streams Processor API

Rain detection uses the Kafka Streams Processor API. The processing topology is:

weather-readings → RainProcessor → rain-alerts

Each incoming record is deserialized into a `WeatherMessage`. Missing or malformed weather records are skipped. A rain alert is generated only when humidity is strictly greater than 70%; a value of exactly 70% does not trigger an alert. For qualifying records, the processor updates a per-station counter in its state store and forwards a `RainAlert`.

The alert includes the station ID, sequence number, timestamp, humidity, accumulated alert count, and the message `It is raining`.

5.2 Processor API Versus DSL

The project also includes an alternative Kafka Streams DSL implementation. The DSL expresses the transformation through stream operations such as `mapValues`, `filter`, and `to`. The Processor API implementation is used for the primary deployment because it provides direct access to the per-station state store and explicit control over record forwarding.

```
(base) ahmedwalaa@ahmeds-air distributed-weather-monitoring-system % docker compose logs --tail=30 rain-processor
rain-processor-1 | [rain-processor] RAIN station=4 s_no=917 humidity=72% (alert #225)
rain-processor-1 | [rain-processor] RAIN station=10 s_no=912 humidity=87% (alert #232)
rain-processor-1 | [rain-processor] RAIN station=7 s_no=916 humidity=83% (alert #237)
rain-processor-1 | [rain-processor] RAIN station=2 s_no=920 humidity=84% (alert #240)
rain-processor-1 | [rain-processor] RAIN station=4 s_no=919 humidity=91% (alert #226)
rain-processor-1 | [rain-processor] RAIN station=7 s_no=918 humidity=98% (alert #238)
rain-processor-1 | [rain-processor] RAIN station=6 s_no=920 humidity=88% (alert #237)
rain-processor-1 | [rain-processor] RAIN station=2 s_no=923 humidity=97% (alert #241)
rain-processor-1 | [rain-processor] RAIN station=1 s_no=1974 humidity=98% (alert #498)
rain-processor-1 | [rain-processor] RAIN station=7 s_no=920 humidity=95% (alert #239)
rain-processor-1 | [rain-processor] RAIN station=5 s_no=922 humidity=96% (alert #264)
rain-processor-1 | [rain-processor] RAIN station=3 s_no=924 humidity=77% (alert #248)
rain-processor-1 | [rain-processor] RAIN station=9 s_no=918 humidity=76% (alert #241)
rain-processor-1 | [rain-processor] RAIN station=1 s_no=1975 humidity=84% (alert #499)
rain-processor-1 | [rain-processor] RAIN station=6 s_no=922 humidity=87% (alert #238)
rain-processor-1 | [rain-processor] RAIN station=8 s_no=920 humidity=98% (alert #256)
rain-processor-1 | [rain-processor] RAIN station=7 s_no=922 humidity=82% (alert #240)
rain-processor-1 | [rain-processor] RAIN station=6 s_no=923 humidity=72% (alert #239)
rain-processor-1 | [rain-processor] RAIN station=8 s_no=922 humidity=84% (alert #257)
rain-processor-1 | [rain-processor] RAIN station=2 s_no=927 humidity=87% (alert #242)
rain-processor-1 | [rain-processor] RAIN station=10 s_no=920 humidity=89% (alert #233)
rain-processor-1 | [rain-processor] RAIN station=4 s_no=926 humidity=85% (alert #227)
rain-processor-1 | [rain-processor] RAIN station=6 s_no=925 humidity=88% (alert #240)
rain-processor-1 | [rain-processor] RAIN station=10 s_no=921 humidity=85% (alert #234)
rain-processor-1 | [rain-processor] RAIN station=6 s_no=926 humidity=79% (alert #241)
rain-processor-1 | [rain-processor] RAIN station=4 s_no=928 humidity=78% (alert #228)
rain-processor-1 | [rain-processor] RAIN station=2 s_no=930 humidity=73% (alert #243)
rain-processor-1 | [rain-processor] RAIN station=4 s_no=929 humidity=94% (alert #229)
rain-processor-1 | [rain-processor] RAIN station=3 s_no=930 humidity=73% (alert #249)
rain-processor-1 | [rain-processor] RAIN station=10 s_no=924 humidity=91% (alert #235)
(base) ahmedwalaa@ahmeds-air distributed-weather-monitoring-system %
```

Figure 6: Rain processor detecting humidity values greater than 70%.

```
(base) ahmedwalaa@ahmeds-air distributed-weather-monitoring-system % docker exec -it $(docker ps -qf name=kafka) \
/opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 \
--topic rain-alerts --max-messages 5
The consumer rebalance protocol (KIP-848) is production-ready! Set group.protocol=consumer to try it out. See https://kafka.apache.org/documentation/#consumer_rebalance_protocol
{"station_id":4,"s_no":1088,"status_timestamp":1787494485,"humidity":88,"alert_count":274,"message":"It is raining"}
{"station_id":3,"s_no":1089,"status_timestamp":1787494485,"humidity":81,"alert_count":297,"message":"It is raining"}
{"station_id":7,"s_no":1090,"status_timestamp":1787494485,"humidity":85,"alert_count":282,"message":"It is raining"}
{"station_id":6,"s_no":1087,"status_timestamp":1787494485,"humidity":76,"alert_count":281,"message":"It is raining"}
{"station_id":2,"s_no":1090,"status_timestamp":1787494485,"humidity":77,"alert_count":292,"message":"It is raining"}
Processed a total of 5 messages
```

Figure 7: Rain-alert message consumed from the Kafka rain-alerts topic.

6 Central Station - Part D

6.1 Concurrent Consumers

The central station initializes the PostgreSQL schema and runs two Kafka consumers in separate threads:

- a weather-reading consumer subscribed to **weather-readings**;
- a rain-alert consumer subscribed to **rain-alerts**.

The consumers use separate JDBC connections and manual Kafka offset commits.

6.2 Batch Persistence

The weather-reading consumer buffers messages and flushes them to PostgreSQL when either the configured threshold of 5,000 records is reached or the 10-second flush interval expires while records are pending.

At the normal rate of approximately ten generated samples per second across all stations, timed batches are usually much smaller than 5,000 records. The batch writer uses a prepared statement and JDBC `executeBatch()` to persist multiple records in a single transaction.

6.3 Delivery Semantics and Idempotency

The persistence sequence is:

1. insert the weather-reading batch into PostgreSQL;
2. commit the PostgreSQL transaction;
3. commit the corresponding Kafka offsets.

This ordering provides at-least-once processing. If a failure occurs after the database transaction commits but before the Kafka offsets commit, the same records can be consumed again. The uniqueness constraint `UNIQUE(station_id, sequence_number)` and an `ON CONFLICT DO NOTHING` insert prevent duplicated historical readings.

6.4 Database Schema

The main table stores all weather readings. A second table archives rain alerts, and the `latest_station_status` view retrieves the newest reading for each station.

Listing 2: Core weather readings schema.

```
1 CREATE TABLE IF NOT EXISTS weather_readings (  
2     id BIGSERIAL PRIMARY KEY,  
3     station_id BIGINT NOT NULL,  
4     sequence_number BIGINT NOT NULL,  
5     battery_status VARCHAR(10) NOT NULL,  
6     timestamp BIGINT NOT NULL,  
7     humidity INT,  
8     temperature INT,  
9     wind_speed INT,  
10    ingested_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
11    CONSTRAINT uq_station_sequence  
12        UNIQUE (station_id, sequence_number)  
13 );
```

```

(base) ahmedwalaa@ahmeds-air distributed-weather-monitoring-system % docker compose logs --tail=30 central-station
central-station-1 | [central] persisted 2 rain alerts
central-station-1 | [central] persisted 1 rain alerts
central-station-1 | [central] persisted 1 rain alerts
central-station-1 | [central] persisted 1 rain alerts
central-station-1 | [central] persisted 1 rain alerts
central-station-1 | [central] persisted 1 rain alerts
central-station-1 | [central] persisted 2 rain alerts
central-station-1 | [central] persisted 2 rain alerts
central-station-1 | [central] batch flushed: 94 rows (total 9480)
central-station-1 | [central] persisted 1 rain alerts
central-station-1 | [central] persisted 3 rain alerts
central-station-1 | [central] persisted 2 rain alerts
central-station-1 | [central] persisted 1 rain alerts
central-station-1 | [central] persisted 1 rain alerts
central-station-1 | [central] persisted 2 rain alerts
central-station-1 | [central] persisted 1 rain alerts
central-station-1 | [central] persisted 1 rain alerts
central-station-1 | [central] persisted 1 rain alerts
central-station-1 | [central] persisted 1 rain alerts
central-station-1 | [central] persisted 2 rain alerts
central-station-1 | [central] persisted 1 rain alerts
central-station-1 | [central] persisted 2 rain alerts
central-station-1 | [central] persisted 1 rain alerts
central-station-1 | [central] persisted 3 rain alerts
central-station-1 | [central] persisted 1 rain alerts
central-station-1 | [central] persisted 2 rain alerts
central-station-1 | [central] batch flushed: 88 rows (total 9568)
central-station-1 | [central] persisted 3 rain alerts
central-station-1 | [central] persisted 1 rain alerts
central-station-1 | [central] persisted 1 rain alerts
(base) ahmedwalaa@ahmeds-air distributed-weather-monitoring-system %

```

Figure 8: Central station consuming Kafka messages and persisting weather readings and rain alerts.

7 Historical Weather Analysis - Part E

SQL queries validate the battery-status distribution, estimate dropped messages, retrieve the latest weather status, and compare rain alerts with qualifying weather readings.

7.1 Battery Status Distribution Per Station

The following query groups records by station and calculates the percentage of low, medium, and high battery states.

Listing 3: Battery distribution query.

```

1 SELECT
2     station_id,
3     COUNT(*) AS total_readings,
4     ROUND(
5         100.0 * COUNT(*) FILTER (
6             WHERE battery_status = 'low'
7         ) / COUNT(*), 2
8     ) AS low_pct,
9     ROUND(
10        100.0 * COUNT(*) FILTER (
11            WHERE battery_status = 'medium'
12        ) / COUNT(*), 2
13    ) AS medium_pct,
14    ROUND(
15        100.0 * COUNT(*) FILTER (
16            WHERE battery_status = 'high'
17        ) / COUNT(*), 2

```



```

18 ) AS high_pct
19 FROM weather_readings
20 GROUP BY station_id
21 ORDER BY station_id;

```

```

(base) ahmedwala@ahmeds-air distributed-weather-monitoring-system % docker exec -it $(docker ps -qf name=postgres) \
psql -U weather -d weather -c "
SELECT station_id,
COUNT(*) AS received,
ROUND(100.0*COUNT(*) FILTER (WHERE battery_status='low')/COUNT(*),2) AS low_pct,
ROUND(100.0*COUNT(*) FILTER (WHERE battery_status='medium')/COUNT(*),2) AS med_pct,
ROUND(100.0*COUNT(*) FILTER (WHERE battery_status='high')/COUNT(*),2) AS high_pct
FROM weather_readings GROUP BY station_id ORDER BY station_id;
"

```

station_id	received	low_pct	med_pct	high_pct
1	1929	28.41	41.37	30.22
2	1020	29.31	39.02	31.67
3	995	29.05	43.12	27.84
4	996	30.02	39.16	30.82
5	1009	33.70	38.16	28.15
6	982	30.35	43.08	26.58
7	983	31.23	38.96	29.81
8	986	32.05	36.71	31.24
9	1008	27.98	38.79	33.23
10	1010	29.01	40.20	30.79

(10 rows)

Figure 9: Measured battery-status distribution for all ten weather stations.

Small deviations from the configured 30% / 40% / 30% probabilities are expected because the readings are generated randomly. Larger sample sizes reduce the relative variation.

7.2 Dropped Messages Per Station

Because the sequence number increases before the drop decision, missing sequence numbers can be estimated by comparing the largest observed sequence number with the number of stored readings.

Listing 4: Dropped-message estimation query.

```

1 SELECT
2     station_id,
3     MAX(sequence_number) AS expected_messages,
4     COUNT(*) AS received_messages,
5     MAX(sequence_number) - COUNT(*) AS dropped_messages,
6     ROUND(
7         100.0 * (MAX(sequence_number) - COUNT(*))
8         / NULLIF(MAX(sequence_number), 0),
9         2
10    ) AS drop_rate_pct
11 FROM weather_readings
12 GROUP BY station_id
13 ORDER BY station_id;

```

```
(base) ahmedwalaa@ahmeds-air distributed-weather-monitoring-system % docker exec -it $(docker ps -qf name=postgres) \
psql -U weather -d weather -c "
SELECT station_id,
       MAX(sequence_number) AS expected,
       COUNT(*) AS received,
       MAX(sequence_number) - COUNT(*) AS dropped,
       ROUND(100.0*(MAX(sequence_number)-COUNT(*)/MAX(sequence_number),2) AS drop_pct
FROM weather_readings GROUP BY station_id ORDER BY station_id;
"

```

station_id	expected	received	dropped	drop_pct
1	2221	1974	247	11.12
2	1171	1066	105	8.97
3	1170	1044	126	10.77
4	1169	1041	128	10.95
5	1169	1056	113	9.67
6	1168	1030	138	11.82
7	1167	1029	138	11.83
8	1166	1034	132	11.32
9	1164	1054	110	9.45
10	1163	1056	107	9.20

(10 rows)

Figure 10: Estimated dropped-message count and drop percentage for each weather station.

The expected drop percentage is approximately 10%. This calculation assumes the stored history includes the beginning of each station's sequence.

7.3 Latest Weather Status Per Station

The latest status for each station can be retrieved directly from PostgreSQL by selecting the largest observed sequence number.

Listing 5: Latest weather status for each station.

```
1 SELECT DISTINCT ON (station_id)
2     station_id,
3     sequence_number,
4     battery_status,
5     humidity,
6     temperature,
7     wind_speed
8 FROM weather_readings
9 ORDER BY station_id, sequence_number DESC;
```

```
(base) ahmedwalaa@ahmeds-air distributed-weather-monitoring-system % docker exec -it $(docker ps -qf name=postgres) \
psql -U weather -d weather -c "
SELECT station_id, sequence_number, battery_status, humidity, temperature, wind_speed
FROM weather_readings w
WHERE sequence_number = (SELECT MAX(sequence_number) FROM weather_readings w2 WHERE w2.station_id = w.station_id)
ORDER BY station_id;
"

```

station_id	sequence_number	battery_status	humidity	temperature	wind_speed
1	2342	high	88	92	0
2	1292	medium	97	76	50
3	1291	low	100	33	42
4	1290	low	67	39	9
5	1289	low	23	106	29
6	1289	medium	25	57	38
7	1288	low	77	20	32
8	1287	low	0	27	39
9	1285	high	30	90	7
10	1284	medium	74	72	33

(10 rows)

Figure 11: Latest weather reading retrieved directly from PostgreSQL for each station.

7.4 Rain Alert Cross-Check

Weather readings are inserted in timed batches, while rain alerts can be persisted sooner. Consequently, a simple comparison of live aggregate counts can temporarily differ even when processing is correct. A more useful cross-check joins older high-humidity readings to rain alerts by station ID and sequence number.

Listing 6: Per-station rain-alert validation.

```
1 WITH eligible_readings AS (  
2     SELECT station_id, sequence_number  
3     FROM weather_readings  
4     WHERE humidity > 70  
5         AND ingested_at < NOW() - INTERVAL '30 seconds'  
6 )  
7 SELECT  
8     r.station_id,  
9     COUNT(*) AS qualifying_readings,  
10    COUNT(a.sequence_number) AS matching_alerts,  
11    COUNT(*) FILTER (  
12        WHERE a.sequence_number IS NULL  
13    ) AS missing_alerts  
14 FROM eligible_readings r  
15 LEFT JOIN rain_alerts a  
16     ON a.station_id = r.station_id  
17     AND a.sequence_number = r.sequence_number  
18 GROUP BY r.station_id  
19 ORDER BY r.station_id;
```



```
(base) ahmedwalaa@192 distributed-weather-monitoring-system % kubectl -n weather exec -it deploy/postgres -- \
[ psql -U weather -d weather
psql (16.15 (Debian 16.15-1.pgdg13+2))
Type "help" for help.

weather=# WITH eligible_readings AS (
        SELECT station_id, sequence_number
        FROM weather_readings
        WHERE humidity > 70
        AND ingested_at < NOW() - INTERVAL '30 seconds'
    )
SELECT
    r.station_id,
    COUNT(*) AS qualifying_readings,
    COUNT(a.sequence_number) AS matching_alerts,
    COUNT(*) FILTER (
        WHERE a.sequence_number IS NULL
    ) AS missing_alerts
FROM eligible_readings r
LEFT JOIN rain_alerts a
    ON a.station_id = r.station_id
    AND a.sequence_number = r.sequence_number
GROUP BY r.station_id
ORDER BY r.station_id;
```

station_id	qualifying_readings	matching_alerts	missing_alerts
1	3614	3614	0
2	3695	3695	0
3	3726	3726	0
4	3708	3708	0
5	3632	3632	0
6	3651	3651	0
7	3646	3646	0
8	3636	3636	0
9	3747	3747	0
10	3757	3757	0

(10 rows)

Figure 12: Per-station cross-check between high-humidity readings and their corresponding rain alerts.

8 Kubernetes Deployment - Part F

8.1 Deployment Environment and Containerization

The system runs on Docker Desktop Kubernetes for macOS in the `weather` namespace. The Weather Station, Rain Processor, and Central Station each use a multi-stage Dockerfile. Maven and Eclipse Temurin JDK 17 build the applications, while the runtime containers contain Eclipse Temurin JRE 17 and the application JAR.

8.2 Kubernetes Resources

The Kubernetes deployment defines independent resources for configuration, ZooKeeper, Kafka, PostgreSQL, persistent database storage, the weather stations, the rain processor, and the central station.

Table 3: Kubernetes resource inventory.

Resource	Role
Namespace <code>weather</code>	Isolates the resources used by the lab deployment.
ConfigMap <code>weather-config</code>	Stores Kafka endpoints, topic names, emission timing, drop rate, batch size, and database URL.
Secret <code>db-credentials</code>	Supplies PostgreSQL credentials to the relevant containers.
ZooKeeper Deployment and Service	Coordinates Kafka through <code>zookeeper:2181</code> .
Kafka Deployment and Service	Exposes the broker to cluster clients at <code>kafka:9092</code> .
Kafka topic-initialization Job	Creates the two application topics with ten partitions.
PostgreSQL Deployment and Service	Provides the relational historical database.
PostgreSQL PV and PVC	Persist database files independently of the PostgreSQL pod.
Weather Station StatefulSet	Runs ten replicas with stable ordinal identities.
Rain Processor Deployment	Executes Kafka Streams rain detection.
Central Station Deployment	Consumes both topics and writes to PostgreSQL.

ZooKeeper and Kafka use `01-zookeeper.yaml` and `02-kafka.yaml`. The PostgreSQL storage resources use `02-postgres-storage.yaml`. The database Deployment and Service are configured in `02-postgres.yaml`.

```
((base) ahmedwala@192 distributed-weather-monitoring-system % kubectl -n weather get pods -o wide)
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
central-station-d8fc9b7df-fxhgt	1/1	Running	0	15m	10.244.0.28	desktop-control-plane	<none>	<none>
kafka-5888746d77-bjzcj	1/1	Running	0	173m	10.244.0.20	desktop-control-plane	<none>	<none>
kafka-topics-init-8fwq2	0/1	Completed	0	173m	10.244.0.21	desktop-control-plane	<none>	<none>
postgres-64cf888fdd-92t9m	1/1	Running	0	17m	10.244.0.27	desktop-control-plane	<none>	<none>
rain-processor-597964d56d-tf7pr	1/1	Running	3 (3h52m ago)	25h	10.244.0.7	desktop-control-plane	<none>	<none>
weather-station-0	1/1	Running	3 (3h52m ago)	25h	10.244.0.11	desktop-control-plane	<none>	<none>
weather-station-1	1/1	Running	3 (3h52m ago)	25h	10.244.0.16	desktop-control-plane	<none>	<none>
weather-station-2	1/1	Running	3 (3h52m ago)	25h	10.244.0.6	desktop-control-plane	<none>	<none>
weather-station-3	1/1	Running	3 (3h52m ago)	25h	10.244.0.8	desktop-control-plane	<none>	<none>
weather-station-4	1/1	Running	3 (3h52m ago)	25h	10.244.0.17	desktop-control-plane	<none>	<none>
weather-station-5	1/1	Running	3 (3h52m ago)	25h	10.244.0.18	desktop-control-plane	<none>	<none>
weather-station-6	1/1	Running	3 (3h52m ago)	25h	10.244.0.15	desktop-control-plane	<none>	<none>
weather-station-7	1/1	Running	3 (3h52m ago)	25h	10.244.0.14	desktop-control-plane	<none>	<none>
weather-station-8	1/1	Running	3 (3h52m ago)	25h	10.244.0.12	desktop-control-plane	<none>	<none>
weather-station-9	1/1	Running	3 (3h52m ago)	25h	10.244.0.13	desktop-control-plane	<none>	<none>
zookeeper-678778b6b6-f26b8	1/1	Running	0	175m	10.244.0.19	desktop-control-plane	<none>	<none>

Figure 13: Kubernetes pods for ZooKeeper, Kafka, PostgreSQL, ten weather stations, the rain processor, and the central station.

```
(base) ahmedwalaa@192 distributed-weather-monitoring-system % kubectl -n weather get \
[ services,deployments,statefulsets,jobs
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/kafka	ClusterIP	10.96.193.32	<none>	9092/TCP	25h
service/postgres	ClusterIP	10.96.136.108	<none>	5432/TCP	25h
service/weather-station	ClusterIP	None	<none>	8080/TCP	25h
service/zookeeper	ClusterIP	10.96.119.153	<none>	2181/TCP	176m

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/central-station	1/1	1	1	25h
deployment.apps/kafka	1/1	1	1	174m
deployment.apps/postgres	1/1	1	1	25h
deployment.apps/rain-processor	1/1	1	1	25h
deployment.apps/zookeeper	1/1	1	1	176m

NAME	READY	AGE
statefulset.apps/weather-station	10/10	25h

NAME	STATUS	COMPLETIONS	DURATION	AGE
job.batch/kafka-topics-init	Complete	1/1	52s	174m

Figure 14: Kubernetes Services, application Deployments, topic-initialization Job, and the ten-replica Weather Station StatefulSet.

8.3 Kafka and ZooKeeper Coordination

Kafka runs from the Bitnami Legacy Kafka 3.6 image, and ZooKeeper uses the official ZooKeeper 3.9 image. The broker connects to zookeeper:2181, and KRaft mode is disabled. Successful broker registration under /brokers/ids/1 confirms that ZooKeeper actively coordinates Kafka.

Listing 7: ZooKeeper-backed Kafka verification.

```
1 kubectl -n weather exec deploy/kafka -- \
2   printenv KAFKA_CFG_ZOOKEEPER_CONNECT
3
4 kubectl -n weather exec deploy/kafka -- \
5   printenv KAFKA_ENABLE_KRAFT
6
7 kubectl -n weather logs deploy/kafka | \
8   grep 'Registered broker'
9
10 kubectl -n weather exec deploy/zookeeper -- \
11   zkCli.sh -server localhost:2181 ls /brokers/ids
```

```
(base) ahmedwala@192 distributed-weather-monitoring-system % kubectl -n weather get pods -l app=zookeeper
NAME                                READY  STATUS   RESTARTS   AGE
zookeeper-678778b6b6-f26b8         1/1    Running   0           152m
(base) ahmedwala@192 distributed-weather-monitoring-system % kubectl -n weather get service zookeeper
NAME      TYPE      CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE
zookeeper ClusterIP   10.96.119.153  <none>         2181/TCP    152m
(base) ahmedwala@192 distributed-weather-monitoring-system % kubectl -n weather exec deploy/kafka -- \
| printenv KAFKA_CFG_ZOOKEEPER_CONNECT
zookeeper:2181
(base) ahmedwala@192 distributed-weather-monitoring-system % kubectl -n weather exec deploy/kafka -- \
| printenv KAFKA_ENABLE_KRAFT
no
(base) ahmedwala@192 distributed-weather-monitoring-system % kubectl -n weather logs deploy/kafka | \
| grep 'Registered broker'
[2026-08-24 13:39:34,650] INFO Registered broker 1 at path /brokers/ids/1 with addresses: PLAINTEXT://kafka.weather.svc.cluster.local:9092, czxid (broker epoch): 25 (kafka.zk.KafkaZkClient)
(base) ahmedwala@192 distributed-weather-monitoring-system % kubectl -n weather exec deploy/zookeeper -- \
| kubectl sh -s -- exec -i kubectl -n weather exec deploy/zookeeper -- \
Connecting to localhost:2181
2026-08-24 16:11:05.427 [myid:] - INFO [main:o.a.z.Environment@98] - Client environment:zookeeper.version=3.9.5-293c895a8d966a3ecb92872be4a1daf87d725da2, built on 2026-08-11 20:18 UTC
2026-08-24 16:11:05.446 [myid:] - INFO [main:o.a.z.Environment@98] - Client environment:host.name=zookeeper-678778b6b6-f26b8
2026-08-24 16:11:05.446 [myid:] - INFO [main:o.a.z.Environment@98] - Client environment:java.version=17.0.28
2026-08-24 16:11:05.446 [myid:] - INFO [main:o.a.z.Environment@98] - Client environment:java.vendor=Eclipse Adoptium
2026-08-24 16:11:05.446 [myid:] - INFO [main:o.a.z.Environment@98] - Client environment:java.home=/opt/java/openjdk
2026-08-24 16:11:05.446 [myid:] - INFO [main:o.a.z.Environment@98] - Client environment:java.class.path=/apache-zookeeper-3.9.5-bin/bin/./zookeeper-metrics-providers/zookeeper-prometheus-me
zookeeper-3.9.5-bin/bin/./build/classes:/apache-zookeeper-3.9.5-bin/bin/./zookeeper-metrics-providers/zookeeper-prometheus-metrics/target/lib/*:/apache-zookeeper-3.9.5-bin/bin/./zooke
-3.9.5-bin/bin/./lib/zookeeper-prometheus-metrics-3.9.5.jar:/apache-zookeeper-3.9.5-bin/bin/./lib/zookeeper-jute-3.9.5.jar:/apache-zookeeper-3.9.5-bin/bin/./lib/zookeeper-3.9.5.jar:/apac
slf4j-api-2.0.13.jar:/apache-zookeeper-3.9.5-bin/bin/./lib/simpleclient-servlet-0.9.0.jar:/apache-zookeeper-3.9.5-bin/bin/./lib/simpleclient-hotspot-0.9.0.jar:/apache-zookeeper-3.9.5-bin/bi
.jar:/apache-zookeeper-3.9.5-bin/bin/./lib/netty-transport-native-unix-common-4.1.130.Final.jar:/apache-zookeeper-3.9.5-bin/bin/./lib/netty-transport-native-epoll-4.1.130.Final-linux-x86_64
che-zookeeper-3.9.5-bin/bin/./lib/netty-transport-4.1.130.Final.jar:/apache-zookeeper-3.9.5-bin/bin/./lib/netty-tcnative-classes-2.0.74.Final.jar:/apache-zookeeper-3.9.5-bin/bin/./lib/nett
ib/netty-tcnative-boringssl-static-2.0.74.Final-linux-aarch64.jar:/apache-zookeeper-3.9.5-bin/bin/./lib/netty-tcnative-boringssl-static-2.0.74.Final-linux-x86_64.jar:/apache-zookeeper-3.9.5-bi
5-bin/bin/./lib/netty-tcnative-boringssl-static-2.0.74.Final-linux-aarch64.jar:/apache-zookeeper-3.9.5-bin/bin/./lib/netty-tcnative-boringssl-static-2.0.74.Final.jar:/apache-zookeeper-3.9.5-bi
andler-4.1.130.Final.jar:/apache-zookeeper-3.9.5-bin/bin/./lib/netty-common-4.1.130.Final.jar:/apache-zookeeper-3.9.5-bin/bin/./lib/netty-codec-4.1.130.Final.jar:/apache-zookeeper-3.9.5-bin
.1.12.1.jar:/apache-zookeeper-3.9.5-bin/bin/./lib/logback-core-1.3.15.jar:/apache-zookeeper-3.9.5-bin/bin/./lib/logback-classic-1.3.15.jar:/apache-zookeeper-3.9.5-bin/bin/./lib/jline-3.25.
per-3.9.5-bin/bin/./lib/jetty-util-9.4.58.v20250814.jar:/apache-zookeeper-3.9.5-bin/bin/./lib/jetty-servlet-9.4.58.v20250814.jar:/apache-zookeeper-3.9.5-bin/bin/./lib/jetty-server-9.4.58.v
zookeeper-3.9.5-bin/bin/./lib/jetty-io-9.4.58.v20250814.jar:/apache-zookeeper-3.9.5-bin/bin/./lib/jetty-http-9.4.58.v20250814.jar:/apache-zookeeper-3.9.5-bin/bin/./lib/javax.servlet-api-3.
.5-bin/bin/./lib/jackson-core-2.15.2.jar:/apache-zookeeper-3.9.5-bin/bin/./lib/jackson-annotations-2.15.2.jar:/apache-zookeeper-3.9.5-bin/bin/./lib/commons-io-2.17.0.jar:/apache-zookeep
ers-0.12.0.jar:/apache-zookeeper-3.9.5-bin/bin/./zookeeper-*.jar:/apache-zookeeper-3.9.5-bin/bin/./zookeeper-server/src/main/resources/lib/*:/usr/lib/*:/usr/lib
2026-08-24 16:11:05.446 [myid:] - INFO [main:o.a.z.Environment@98] - Client environment:java.library.path=/usr/java/packages/lib:/usr/lib64:/lib64:/lib:/usr/lib
2026-08-24 16:11:05.447 [myid:] - INFO [main:o.a.z.Environment@98] - Client environment:java.io.tmpdir=/tmp
2026-08-24 16:11:05.447 [myid:] - INFO [main:o.a.z.Environment@98] - Client environment:java.compiler=<NA>
2026-08-24 16:11:05.447 [myid:] - INFO [main:o.a.z.Environment@98] - Client environment:os.name=Linux
2026-08-24 16:11:05.447 [myid:] - INFO [main:o.a.z.Environment@98] - Client environment:os.arch=aarch64
2026-08-24 16:11:05.447 [myid:] - INFO [main:o.a.z.Environment@98] - Client environment:os.version=6.12.76-linuxkit
2026-08-24 16:11:05.447 [myid:] - INFO [main:o.a.z.Environment@98] - Client environment:user.name=poot
2026-08-24 16:11:05.447 [myid:] - INFO [main:o.a.z.Environment@98] - Client environment:user.home=/root
2026-08-24 16:11:05.447 [myid:] - INFO [main:o.a.z.Environment@98] - Client environment:user.dir=/apache-zookeeper-3.9.5-bin
2026-08-24 16:11:05.447 [myid:] - INFO [main:o.a.z.Environment@98] - Client environment:os.memory.free=40MB
2026-08-24 16:11:05.447 [myid:] - INFO [main:o.a.z.Environment@98] - Client environment:os.memory.max=247MB
2026-08-24 16:11:05.447 [myid:] - INFO [main:o.a.z.Environment@98] - Client environment:os.memory.total=11MB
2026-08-24 16:11:05.449 [myid:] - INFO [main:o.a.z.ZooKeeper@9635] - Initiating client connection, connectString=localhost:2181 sessionTimeout=30000 watcher=org.apache.zookeeper.ZooKeeperMain
2026-08-24 16:11:05.450 [myid:] - INFO [main:o.a.z.X509Util@1093] - Setting -D java.tls.rejectClientInitiatedRenegotiation=true to disable client-initiated TLS renegotiation
2026-08-24 16:11:05.882 [myid:] - INFO [main:o.a.z.c.X509Util@1015] - Default TLS protocol is TLSv1.3, supported TLS protocols are [TLSv1.3, TLSv1.2, TLSv1.1, TLSv1, SSLv3, SSLv2Hello]
2026-08-24 16:11:05.922 [myid:] - INFO [main:o.a.z.ClientCnxnSocket@235] - jute.maxbuffer value is 1048575 Bytes
2026-08-24 16:11:05.943 [myid:] - INFO [main:o.a.z.ClientCnxn@1734] - zookeeper.request.timeout value is 0, feature enabled=false
2026-08-24 16:11:05.950 [myid:localhost:2181] - INFO [main-SendThread(localhost:2181):o.a.z.ClientCnxn$SendThread@1170] - Opening socket connection to server localhost/[0:0:0:0:0:0:0:1]:2181
2026-08-24 16:11:05.950 [myid:localhost:2181] - INFO [main-SendThread(localhost:2181):o.a.z.ClientCnxn$SendThread@1172] - SASL config status: Will not attempt to authenticate using SASL (unk
2026-08-24 16:11:05.957 [myid:localhost:2181] - INFO [main-SendThread(localhost:2181):o.a.z.ClientCnxn$SendThread@1066] - Socket connection established, initiating session, client: [/0:0:0:0:0:0:0:1]
2026-08-24 16:11:06.157 [myid:localhost:2181] - INFO [main-SendThread(localhost:2181):o.a.z.ClientCnxn$SendThread@1438] - Session establishment complete on server localhost/[0:0:0:0:0:0:0:1]:2181
WATCHER::
WatchedEvent state:SyncConnected type:None path:null zxid: -1
[]
2026-08-24 16:11:06.168 [myid:] - INFO [main:o.a.z.u.ServiceUtils@45] - Exiting JVM with code 0
(base) ahmedwala@192 distributed-weather-monitoring-system %
```

Figure 15: Kafka ZooKeeper connection and successful registration of broker 1.

8.4 Weather Station StatefulSet

The Weather Station StatefulSet provides stable pod names from `weather-station-0` through `weather-station-9`. The application maps the pod ordinal to logical station IDs 1 through 10, avoiding ten separate Deployment manifests. A headless Service supports the StatefulSet's network identities.

8.5 PostgreSQL Persistent Storage

PostgreSQL mounts the `weather-postgres-pvc` claim at `/var/lib/postgresql/data`. The claim is bound to the 5 GiB `weather-postgres-pv` PersistentVolume, which uses `ReadWriteOnce` access and a `Retain` reclaim policy.

The PersistentVolume is backed by a local `hostPath` on the Docker Desktop Kubernetes node. This preserves database files across PostgreSQL pod replacement on that node. It does not provide protection against deleting or resetting the complete Kubernetes cluster.

Listing 8: PostgreSQL persistent-storage verification.

```
1 kubectl get pv weather-postgres-pv
2
3 kubectl -n weather get pvc weather-postgres-pvc
4
5 kubectl -n weather get deployment postgres \
6   -o jsonpath='{.spec.template.spec.volumes[0].persistentVolumeClaim.claimName}'
```

```
(base) ahmedwala@192 distributed-weather-monitoring-system % kubectl get pv weather-postgres-pv \
-o custom-columns='NAME:.metadata.name,CAPACITY:.spec.capacity.storage,ACCESS:.spec.accessModes[0],RECLAIM:.spec.persistentVolumeReclaimPolicy,STATUS:.status
.phase,CLAIM:.spec.claimRef.name'
NAME          CAPACITY  ACCESS  RECLAIM  STATUS  CLAIM
weather-postgres-pv  5Gi      ReadWriteOnce  Retain   Bound   weather-postgres-pvc
(base) ahmedwala@192 distributed-weather-monitoring-system % kubectl -n weather get pvc weather-postgres-pvc \
-o custom-columns='NAME:.metadata.name,STATUS:.status.phase,VOLUME:.spec.volumeName,CAPACITY:.spec.capacity.storage,ACCESS:.status.accessModes[0]'
NAME          STATUS  VOLUME  CAPACITY  ACCESS
weather-postgres-pvc  Bound   weather-postgres-pv  5Gi      ReadWriteOnce
(base) ahmedwala@192 distributed-weather-monitoring-system % kubectl -n weather get deployment postgres \
-o jsonpath='{.PVC={.spec.template.spec.volumes[0].persistentVolumeClaim.claimName}{"\n"}MOUNT={.spec.template.spec.containers[0].volumeMounts[0].mountPath}{"\n"}'
PVC=weather-postgres-pvc
MOUNT=/var/lib/postgresql/data
(base) ahmedwala@192 distributed-weather-monitoring-system %
```

Figure 16: Bound PostgreSQL PersistentVolume and PersistentVolumeClaim mounted by the PostgreSQL Deployment.

8.6 Database Persistence Verification

Database persistence was tested by temporarily stopping the central station, recording the number of rows in both PostgreSQL tables, replacing the PostgreSQL pod, and repeating the same queries after the replacement pod became ready.

Table 4: PostgreSQL row counts before and after pod replacement.

Database table	Before restart	After restart
weather_readings	114,804	114,804
rain_alerts	34,155	34,155

Both counts remained unchanged, demonstrating that the replacement PostgreSQL pod reused the existing PersistentVolumeClaim. After the central station was scaled back to one replica, the number of weather readings continued to increase, confirming that normal ingestion resumed.

```
(base) ahmedwala@192 distributed-weather-monitoring-system % kubectl -n weather scale deployment/central-station --replicas=0
deployment.apps/central-station scaled
(base) ahmedwala@192 distributed-weather-monitoring-system % kubectl -n weather get pods -l app=central-station
No resources found in weather namespace.
(base) ahmedwala@192 distributed-weather-monitoring-system % kubectl -n weather get pod \
-l app=postgres \
-o jsonpath='{.items[0].metadata.name}'
(base) ahmedwala@192 distributed-weather-monitoring-system % kubectl -n weather exec deploy/postgres -- \
psql -U weather -d weather -t -F, \
-c "SELECT COUNT(*) FROM weather_readings UNION ALL SELECT COUNT(*) FROM rain_alerts;"
> /tmp/weather-db-before.txt
(base) ahmedwala@192 distributed-weather-monitoring-system % cat /tmp/weather-db-before.txt
weather_readings,114804
rain_alerts,34155
(base) ahmedwala@192 distributed-weather-monitoring-system % kubectl -n weather rollout restart deployment/postgres
deployment.apps/postgres restarted
(base) ahmedwala@192 distributed-weather-monitoring-system % kubectl -n weather rollout status \
deployment/postgres \
--timeout=240s
Waiting for deployment "postgres" rollout to finish: 0 of 1 updated replicas are available...
deployment "postgres" successfully rolled out
(base) ahmedwala@192 distributed-weather-monitoring-system % kubectl -n weather get pod \
-l app=postgres \
-o jsonpath='{.items[0].metadata.name}'
(base) ahmedwala@192 distributed-weather-monitoring-system % kubectl -n weather exec deploy/postgres -- \
psql -U weather -d weather -t -F, \
-c "SELECT COUNT(*) FROM weather_readings UNION ALL SELECT COUNT(*) FROM rain_alerts;"
> /tmp/weather-db-after.txt
(base) ahmedwala@192 distributed-weather-monitoring-system %
(base) ahmedwala@192 distributed-weather-monitoring-system % echo "Old PostgreSQL pod: $OLD_PG_POD"
Old PostgreSQL pod: postgres-cd56f4f5b-62d98
(base) ahmedwala@192 distributed-weather-monitoring-system % echo "New PostgreSQL pod: $NEW_PG_POD"
New PostgreSQL pod: postgres-64cf888fd-9249m
(base) ahmedwala@192 distributed-weather-monitoring-system % printf "%-20s %-15s %-15s\n" TABLE BEFORE AFTER
TABLE      BEFORE      AFTER
(base) ahmedwala@192 distributed-weather-monitoring-system % paste -d, \
/tmp/weather-db-before.txt \
/tmp/weather-db-after.txt | \
awk -F, '{printf "%-20s %-15s %-15s\n",$1,$2,$4}'
weather_readings    114804      114804
rain_alerts         34155       34155
(base) ahmedwala@192 distributed-weather-monitoring-system % cmp -s \
/tmp/weather-db-before.txt \
/tmp/weather-db-after.txt
&& echo "PASS: PostgreSQL data survived pod replacement"
// echo "FAIL: database counts changed"
PASS: PostgreSQL data survived pod replacement
(base) ahmedwala@192 distributed-weather-monitoring-system %
```

Figure 17: PostgreSQL pod-replacement test confirming that weather readings and rain alerts survived the restart.

8.7 Configuration and Secrets

Application settings are supplied through the `weather-config` ConfigMap. The configured emission interval is 1,000 ms, the drop rate is 0.10, the batch threshold is 5,000 records, and the flush interval is 10,000 ms. Database credentials are injected from the `db-credentials` Kubernetes Secret rather than being hardcoded in application source files.

```

[(base) ahmedwalaa@192 distributed-weather-monitoring-system % kubectl -n weather describe configmap weather-config
Name:      weather-config
Namespace: weather
Labels:    <none>
Annotations: <none>

Data
====
BATCH_SIZE:
-----
5000

DB_URL:
-----
jdbc:postgresql://postgres:5432/weather

DROP_RATE:
-----
0.10

EMIT_INTERVAL_MS:
-----
1000

FLUSH_INTERVAL_MS:
-----
10000

KAFKA_BOOTSTRAP_SERVERS:
-----
kafka:9092

TOPIC_RAIN:
-----
rain-alerts

TOPIC_WEATHER:
-----
weather-readings

BinaryData
=====
Events: <none>
[(base) ahmedwalaa@192 distributed-weather-monitoring-system % kubectl -n weather describe secret db-credentials
Name:      db-credentials
Namespace: weather
Labels:    <none>
Annotations: <none>

Type: Opaque

Data
====
DB_PASSWORD: 11 bytes
DB_USER:     7 bytes
[(base) ahmedwalaa@192 distributed-weather-monitoring-system % kubectl -n weather describe deployment postgres \
| sed -n '/Environment:/,/Mounts:/p'
Environment:
  POSTGRES_DB:      weather
  POSTGRES_USER:    <set to the key 'DB_USER' in secret 'db-credentials'>    Optional: false
  POSTGRES_PASSWORD: <set to the key 'DB_PASSWORD' in secret 'db-credentials'> Optional: false
Mounts:

```

Figure 18: Kubernetes ConfigMap values and database Secret metadata without exposing credential values.

8.8 Runtime Evidence

The following screenshots show the application operating inside Kubernetes: weather readings are generated, qualifying humidity values produce alerts, the central station

persists batches, and the database can be queried from the cluster.

```
(base) ahmedwalaa@ahmeds-air distributed-weather-monitoring-system % kubectl -n weather logs weather-station-0 --tail=15
[station-1] sent=10290 dropped=1157 last={"station_id":1,"s_no":11427,"battery_status":"medium","status_timestamp":1787508930,"weather":{"humidity":87,"temperature":96,"wind_speed":34}}
[station-1] dropped s_no=11448 (total dropped=1148)
[station-1] sent=10320 dropped=1143 last={"station_id":1,"s_no":11463,"battery_status":"medium","status_timestamp":1787508966,"weather":{"humidity":40,"temperature":66,"wind_speed":3}}
[station-1] sent=10350 dropped=1146 last={"station_id":1,"s_no":11496,"battery_status":"medium","status_timestamp":1787508999,"weather":{"humidity":62,"temperature":52,"wind_speed":37}}
[station-1] dropped s_no=11523 (total dropped=1150)
[station-1] sent=10380 dropped=1152 last={"station_id":1,"s_no":11532,"battery_status":"low","status_timestamp":1787509030,"weather":{"humidity":4,"temperature":86,"wind_speed":28}}
[station-1] sent=10410 dropped=1157 last={"station_id":1,"s_no":11567,"battery_status":"medium","status_timestamp":1787509073,"weather":{"humidity":36,"temperature":63,"wind_speed":4}}
[station-1] sent=10440 dropped=1157 last={"station_id":1,"s_no":11597,"battery_status":"high","status_timestamp":1787509103,"weather":{"humidity":90,"temperature":112,"wind_speed":22}}
[station-1] sent=10470 dropped=1159 last={"station_id":1,"s_no":11629,"battery_status":"medium","status_timestamp":1787509135,"weather":{"humidity":19,"temperature":28,"wind_speed":38}}
[station-1] dropped s_no=11634 (total dropped=1160)
[station-1] sent=10500 dropped=1161 last={"station_id":1,"s_no":11661,"battery_status":"low","status_timestamp":1787509167,"weather":{"humidity":91,"temperature":77,"wind_speed":0}}
[station-1] sent=10530 dropped=1163 last={"station_id":1,"s_no":11693,"battery_status":"high","status_timestamp":1787509200,"weather":{"humidity":59,"temperature":50,"wind_speed":7}}
[station-1] sent=10560 dropped=1163 last={"station_id":1,"s_no":11723,"battery_status":"high","status_timestamp":1787509230,"weather":{"humidity":60,"temperature":22,"wind_speed":14}}
[station-1] sent=10590 dropped=1166 last={"station_id":1,"s_no":11756,"battery_status":"low","status_timestamp":1787509263,"weather":{"humidity":12,"temperature":110,"wind_speed":38}}
[station-1] dropped s_no=11774 (total dropped=1170)
(base) ahmedwalaa@ahmeds-air distributed-weather-monitoring-system %
```

Figure 19: Weather station pod logs showing generated readings and simulated message drops.

```
(base) ahmedwalaa@ahmeds-air distributed-weather-monitoring-system % kubectl -n weather logs deploy/rain-processor --tail=20
[rain-processor] RAIN station=5 s_no=11731 humidity=73% (alert #3172)
[rain-processor] RAIN station=3 s_no=11730 humidity=96% (alert #3186)
[rain-processor] RAIN station=4 s_no=11732 humidity=80% (alert #3113)
[rain-processor] RAIN station=1 s_no=11731 humidity=86% (alert #3080)
[rain-processor] RAIN station=6 s_no=11731 humidity=94% (alert #3111)
[rain-processor] RAIN station=10 s_no=11734 humidity=86% (alert #3128)
[rain-processor] RAIN station=2 s_no=11731 humidity=77% (alert #3195)
[rain-processor] RAIN station=9 s_no=11735 humidity=73% (alert #3041)
[rain-processor] RAIN station=3 s_no=11733 humidity=98% (alert #3187)
[rain-processor] RAIN station=1 s_no=11734 humidity=71% (alert #3081)
[rain-processor] RAIN station=5 s_no=11735 humidity=71% (alert #3173)
[rain-processor] RAIN station=2 s_no=11734 humidity=75% (alert #3196)
[rain-processor] RAIN station=4 s_no=11737 humidity=77% (alert #3114)
[rain-processor] RAIN station=6 s_no=11736 humidity=85% (alert #3112)
[rain-processor] RAIN station=2 s_no=11736 humidity=96% (alert #3197)
[rain-processor] RAIN station=7 s_no=11737 humidity=87% (alert #3080)
[rain-processor] RAIN station=1 s_no=11739 humidity=91% (alert #3082)
[rain-processor] RAIN station=3 s_no=11739 humidity=100% (alert #3188)
[rain-processor] RAIN station=8 s_no=11740 humidity=96% (alert #3176)
[rain-processor] RAIN station=4 s_no=11741 humidity=72% (alert #3115)
(base) ahmedwalaa@ahmeds-air distributed-weather-monitoring-system %
```

Figure 20: Rain Processor running in Kubernetes and detecting humidity values greater than 70%.

```
(base) ahmedwalaa@ahmeds-air distributed-weather-monitoring-system % kubectl -n weather logs deploy/central-station --tail=20
[central] persisted 1 rain alerts
[central] persisted 2 rain alerts
[central] persisted 1 rain alerts
[central] batch flushed: 94 rows (total 105300)
[central] persisted 5 rain alerts
[central] persisted 1 rain alerts
[central] persisted 1 rain alerts
[central] persisted 4 rain alerts
[central] persisted 2 rain alerts
[central] persisted 1 rain alerts
[central] persisted 1 rain alerts
[central] persisted 4 rain alerts
[central] persisted 1 rain alerts
[central] persisted 1 rain alerts
[central] persisted 1 rain alerts
[central] persisted 1 rain alerts
[central] persisted 1 rain alerts
[central] persisted 2 rain alerts
[central] batch flushed: 96 rows (total 105396)
[central] persisted 5 rain alerts
(base) ahmedwalaa@ahmeds-air distributed-weather-monitoring-system %
```

Figure 21: Central Station logs showing JDBC batch persistence inside Kubernetes.


```
(base) ahmed@ahmeds-air distributed-weather-monitoring-system % kubectl -n weather exec deploy/postgres -- psql -U weather -d weather -c "SELECT station_id, COUNT(*) AS received, MAX(sequence_number) AS expected FROM weather_readings GROUP BY station_id ORDER BY station_id;"
```

station_id	received	expected
1	11909	11909
2	11908	11908
3	11909	11909
4	11911	11911
5	11910	11910
6	11908	11908
7	11910	11910
8	11910	11910
9	11910	11910
10	11911	11911

(10 rows)

Figure 22: PostgreSQL query results obtained from the Kubernetes deployment.

9 Challenges and Lessons Learned

The implementation and deployment demonstrated several practical distributed-system concerns:

- **Kafka coordination must match the deployment requirement.** The original Kafka setup used KRaft mode. Replacing it with a ZooKeeper-compatible broker and verifying broker registration satisfied the required Kafka-plus-ZooKeeper architecture.
- **Container image availability affects reproducibility.** The original Bitnami image tags were unavailable. The replacement images are the official ZooKeeper 3.9 image and Bitnami Legacy Kafka 3.6.
- **Persistent storage requires explicit migration.** Existing PostgreSQL data was backed up, restored into a PVC-backed database, and verified before and after pod replacement.
- **Random simulation must remain measurable.** Incrementing `s_no` before the drop decision makes missing samples observable through sequence-number gaps.
- **Partition keys preserve per-station ordering.** Using `station_id` as the Kafka key keeps records from the same station in a consistent partition, although different stations can share a partition.
- **Database writes and offset commits must be coordinated.** Committing PostgreSQL before Kafka offsets, combined with a uniqueness constraint, supports idempotent recovery.
- **Batching balances throughput and latency.** A 5,000-record threshold and a 10-second flush interval reduce database I/O without indefinitely delaying low-volume traffic.
- **Kubernetes resources have distinct responsibilities.** StatefulSets provide stable station identities, Secrets protect credentials, and PersistentVolumes preserve database contents across pod replacement.

10 Conclusion

The project implements the required distributed weather monitoring system using ten simulated weather stations, Apache Kafka, ZooKeeper, a Kafka Streams rain processor, a central station, and PostgreSQL. Each station generates one sample per second, applies

the required battery-status probabilities, and intentionally drops approximately 10% of generated samples.

Weather readings are published to Kafka, and readings with humidity greater than 70% generate alerts on a separate topic. The central station consumes both streams and persists their records in PostgreSQL using batched database writes. SQL analysis validates battery distributions, estimates dropped messages, retrieves the latest station status, and checks the relationship between qualifying readings and generated alerts.

The complete system is deployed on Docker Desktop Kubernetes. Kafka operates with an active ZooKeeper service, and PostgreSQL uses a 5 GiB PersistentVolume and PersistentVolumeClaim. A controlled PostgreSQL pod-replacement test preserved all 114,804 weather readings and 34,155 rain alerts, demonstrating that database contents remain available across the lifecycle of the PostgreSQL pod.